$x[n-1]$ is point 27, etc. To understand the algorithm being used, we ask ourselves: "What information was available to the professor to calculate $y[n]$, the sample currently being worked on?"

The most obvious source of information is the *input signal*, that is, the values: $x[n]$, $x[n-1]$, $x[n-2]$, $\cdots$. The professor could have been multiplying each point in the input signal by a coefficient, and adding the products together:

$$y[n] = a_0 x[n] + a_1 x[n-1] + a_2 x[n-2] + a_3 x[n-3] + \cdots$$

You should recognize that this is nothing more than simple convolution, with the coefficients: a_0, a_1, a_2, $\cdots$, forming the convolution kernel. If this was all the professor was doing, there wouldn't be much need for this story, or this chapter. However, there is another source of information that the professor had access to: the *previously* calculated values of the output signal, held in: $y[n-1]$, $y[n-2]$, $y[n-3]$, $\cdots$. Using this additional information, the algorithm would be in the form:

$$y[n] = a_0 x[n] + a_1 x[n-1] + a_2 x[n-2] + a_3 x[n-3] + \cdots + b_1 y[n-1] + b_2 y[n-2] + b_3 y[n-3] + \cdots$$

EQUATION 19-1
The recursion equation. In this equation, $x[\ ]$ is the input signal, $y[\ ]$ is the output signal, and the a's and b's are coefficients.

In words, each point in the output signal is found by multiplying the values from the input signal by the "a" coefficients, multiplying the previously calculated values from the output signal by the "b" coefficients, and adding the products together. Notice that there isn't a value for b_0, because this corresponds to the sample being calculated. Equation 19-1 is called the **recursion equation**, and filters that use it are called **recursive filters**. The "a" and "b" values that define the filter are called the **recursion coefficients**. In actual practice, no more than about a dozen recursion coefficients can be used or the filter becomes unstable (i.e., the output continually increases or oscillates). Table 19-1 shows an example recursive filter program.

Recursive filters are useful because they *bypass* a longer convolution. For instance, consider what happens when a delta function is passed through a recursive filter. The output is the filter's *impulse response*, and will typically be a sinusoidal oscillation that exponentially decays. Since this impulse response in infinitely long, recursive filters are often called *infinite impulse response* (IIR) filters. In effect, recursive filters *convolve* the input signal with a very long filter kernel, although only a few coefficients are involved.